

Mathematics for Deep Learning

Rudy Kusdiantara

Industrial and Financial Mathematics Research Group



Bandung, 7 July 2026

Introduction

This note provides a summary that can be used as an introduction to the mathematical background behind an Artificial Neural Network model. The most important material is focused on the discussion of the mathematics for *backpropagation*. This is where the learning process of a machine occurs, manifested in the form of iterative changes to parameter values. In essence, *backpropagation* is the chain rule of derivatives that leads to the necessary conditions for optimization problems.

There is a consistent pattern in the mathematical notation for *backpropagation*. This pattern has been intentionally made dominant to aid learning, especially when used later to study *backpropagation* in other Deep Learning models. The notations in this note will be closely aligned with Python code.

Roadmap (Theory Session)

- ① **Perceptron** – the basic building block of a neural network
- ② **Multilayer Perceptron (MLP)** – forward & backward propagation
- ③ **Convolutional Neural Network (CNN)** – for image data
- ④ **Recurrent Neural Network (RNN) & LSTM** – for sequential data
- ⑤ **Physics-Informed Neural Networks (PINN)** – solving ODEs with neural networks
- ⑥ **When to use which architecture?** – practical summary

Note

After this theory session, we will continue with a **2-hour live coding session** (Python, PyTorch/Keras) to put these concepts into practice.

Comparison of Activation Functions

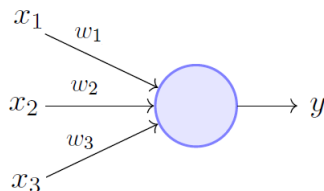
To simplify the discussion, only the Sigmoid activation function is used as an illustration. There are advantages and disadvantages to using the Sigmoid function as an activation function.

Below is a table of various other activation functions.

Aspect	Sigmoid	Tanh	ReLU	Leaky ReLU
Formula	$\sigma(x) = \frac{1}{1 + e^{-x}}$	$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$	$f(x) = \max(0, x)$	$f(x) = \max(\alpha x, x)$ (e.g., $\alpha = 0.01$)
Range	(0, 1)	(-1, 1)	$[0, \infty)$	$(-\infty, \infty)$
Nonlinearity	Yes	Yes	Yes	Yes
Derivative	$\sigma(x)(1 - \sigma(x))$	$1 - \tanh^2(x)$	0 or 1	α or 1
Vanishing Gradient	Yes	Yes	No	No
Exploding Gradient	No	No	Possible	Possible
Sparsity	No	No	Yes	Yes
Typical Use Cases	Output layer (binary classification)	RNNs (historically)	CNNs, MLPs (hidden layers)	Improved ReLU robustness

Perceptron Model

The **Perceptron Model** for a neuron is visually shown in the figure below:



Perceptron Model (Minsky-Papert in 1969)

- The perceptron is a simple model for a neuron in an artificial neural network.
- It consists of three inputs (x_1, x_2, x_3), each associated with weights (w_1, w_2, w_3).
- The output of the perceptron is determined based on a linear combination of the inputs and their weights.

Mathematical Structure of the Perceptron

- In general, given input data as a matrix $\mathbf{X}$ of size $T \times K$ (T = number of observations/samples, K = number of features).
- The output data is a vector y of size $T \times 1$.
- As an illustration used throughout this note, for $T = 4$ and $K = 3$:

$$\mathbf{X} = \begin{bmatrix} x_{11} & x_{12} & x_{13} \\ x_{21} & x_{22} & x_{23} \\ x_{31} & x_{32} & x_{33} \\ x_{41} & x_{42} & x_{43} \end{bmatrix}, \quad y = \begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ y_4 \end{bmatrix} \quad (1)$$

Example Input and Output Data

The input data $\mathbf{X}$ consists of three *variables (features)* and four observations. Values of x_{ti} and y_t are 0 or 1.

Example	Input			Output
Example 1	0	0	1	0
Example 2	1	1	1	1
Example 3	1	0	1	1
Example 4	0	1	1	0
New situation	1	0	0	?

The Perceptron model tries to find a rule (function) linking input and output, so that the predicted output $\hat{y}$ is as close as possible to the actual output $\mathbf{y}$.

Forward Propagation

First, a linear combination of the columns of $\mathbf{X}$ is formed for each observation:

$$\mathbf{z} = \mathbf{X}\mathbf{w}, \quad \mathbf{w} = \begin{bmatrix} w_1 \\ w_2 \\ w_3 \end{bmatrix} \quad (2)$$

Explicitly, for $t = 1, \dots, 4$:

$$z_t = x_{t1}w_1 + x_{t2}w_2 + x_{t3}w_3. \quad (3)$$

Activation Function

The value $\mathbf{z}$ is transformed by the Sigmoid activation function into the *predicted output* $\hat{y}_t$:

$$\hat{y}_t = \sigma(z_t) = \frac{1}{1 + e^{-z_t}}, \quad t = 1, 2, 3, 4. \quad (4)$$

A key property used repeatedly in backpropagation is the derivative of the Sigmoid function:

$$\frac{\partial \hat{y}_t}{\partial z_t} = \hat{y}_t(1 - \hat{y}_t). \quad (5)$$

This compact form – the output multiplied by one minus itself – is why Sigmoid is convenient for hand-derived gradients.

Loss Function and Cost Function

To measure prediction accuracy, the error e_t is:

$$e_t = y_t - \hat{y}_t \quad (6)$$

Since errors can be positive or negative and would cancel out if summed directly, a *loss function* (squared error) is used instead:

$$L_t = \frac{1}{2}e_t^2 = \frac{1}{2}(y_t - \hat{y}_t)^2. \quad (7)$$

The total *cost function* C sums the loss over all $T = 4$ observations:

$$C = \sum_{t=1}^4 L_t = \frac{1}{2} \mathbf{e}^T \mathbf{e}, \quad \mathbf{e} = \mathbf{y} - \hat{\mathbf{y}}. \quad (8)$$

The quadratic form penalizes large errors more heavily than small ones, which stabilizes optimization.

Backward Propagation

Initially, with a given $\mathbf{w}$, the cost function C may be high – the machine's prediction $\hat{\mathbf{y}}$ is still far from $\mathbf{y}$. *Learning* means adjusting $\mathbf{w}$ so that C decreases.

The machine learning process is therefore an optimization problem:

$$\min_{\mathbf{w}} C = \min_{\mathbf{w}} \frac{1}{2} \mathbf{e}^T \mathbf{e}. \quad (9)$$

Gradient Descent for Parameter Optimization

One of the most commonly used optimization methods in Deep Learning models is **gradient descent**, with the update rule for the parameters $\mathbf{w}$ expressed as:

$$\mathbf{w}^{(n+1)} = \mathbf{w}^{(n)} - \alpha \left. \frac{\partial \mathcal{C}}{\partial \mathbf{w}} \right|_{\mathbf{w}^{(n)}} \quad (10)$$

$$\begin{bmatrix} w_1^{(n+1)} \\ w_2^{(n+1)} \\ w_3^{(n+1)} \end{bmatrix} = \begin{bmatrix} w_1^{(n)} \\ w_2^{(n)} \\ w_3^{(n)} \end{bmatrix} - \alpha \begin{bmatrix} \left. \frac{\partial \mathcal{C}}{\partial w_1} \right|_{w_1^{(n)}} \\ \left. \frac{\partial \mathcal{C}}{\partial w_2} \right|_{w_2^{(n)}} \\ \left. \frac{\partial \mathcal{C}}{\partial w_3} \right|_{w_3^{(n)}} \end{bmatrix} \quad (11)$$

where $\alpha > 0$ represents the *learning rate* and $\mathbf{w}^{(0)}$ are the *initial values* for $\mathbf{w}$. This equation represents the update rule for each w_k : if the gradient $\frac{\partial \mathcal{C}}{\partial w_k}$ is positive (negative), then the value of w_k at iteration n will be decreased (increased) to obtain the value $w_k^{(n+1)}$ at iteration $n + 1$. When the iterations converge, the gradient will be zero (or nearly zero).

Intuition: Why Follow the Negative Gradient?

- Think of $C(\mathbf{w})$ as a landscape (a valley), and $\mathbf{w}$ as your current position.
- The gradient $\frac{\partial C}{\partial \mathbf{w}}$ points in the direction of **steepest increase** of C .
- Moving in the **opposite** direction ($-\alpha \frac{\partial C}{\partial \mathbf{w}}$) is the fastest local way to **decrease** C – like walking downhill.
- α (learning rate) controls the step size: too small $\Rightarrow$ slow learning; too large $\Rightarrow$ overshooting/instability.
- Applying the chain rule to compute $\frac{\partial C}{\partial \mathbf{w}}$ through every layer of the network is exactly what **backpropagation** does.

Gradient Descent Algorithm for the Perceptron

Applying the chain rule to $\frac{\partial C}{\partial \mathbf{w}}$ (using the Sigmoid derivative from before) gives a compact matrix form:

$$\mathbf{w}^{(n+1)} = \mathbf{w}^{(n)} - \alpha \mathbf{X}^T \boldsymbol{\delta}^{(n)}, \quad \boldsymbol{\delta}^{(n)} = -\mathbf{e}^{(n)} \circ \hat{\mathbf{y}}^{(n)} \circ (1 - \hat{\mathbf{y}}^{(n)}) \quad (12)$$

where $\circ$ is the element-wise (Hadamard) product. In words: at every iteration n ,

- 1 compute the prediction $\hat{\mathbf{y}}^{(n)}$ and error $\mathbf{e}^{(n)}$ (forward pass);
- 2 compute $\boldsymbol{\delta}^{(n)}$, the “local error signal” at the output;
- 3 update $\mathbf{w}$ using $\mathbf{X}^T \boldsymbol{\delta}^{(n)}$ (backward pass).

This forward–backward loop, repeated many times, **is** the training process.

Multilayer Perceptron (MLP) Model

The multilayer perceptron (MLP) model in Andrew Trask's work can be shown in the figure below:

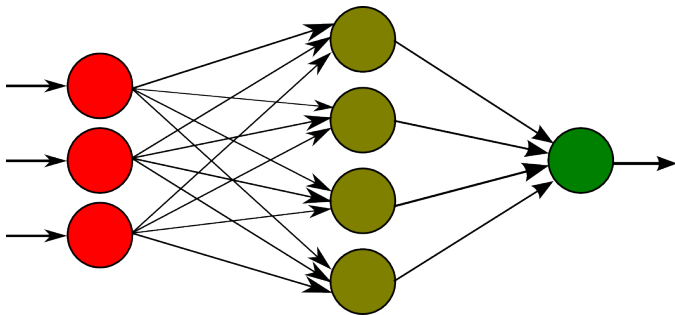


Figure: Multilayer Perceptron (MLP)

Forward Propagation – Hidden Layer

In the **hidden layer**, there are two perceptron models expressed through one matrix multiplication:

$$\mathbf{Z}_{T \times H}^{(1)} = \mathbf{X}_{T \times K} \mathbf{W}_{K \times H}^{(0)}, \quad z_{th}^{(1)} = x_{t1}w_{1h}^{(0)} + x_{t2}w_{2h}^{(0)} + x_{t3}w_{3h}^{(0)} \quad (13)$$

for $t = 1, \dots, 4$ (samples) and $h = 1, 2$ (hidden neurons). Exactly as in the perceptron, this is followed by an activation:

$$\mathbf{A}^{(1)} = \sigma(\mathbf{Z}^{(1)}), \quad a_{th}^{(1)} = \sigma(z_{th}^{(1)}). \quad (14)$$

Note: the only difference from a single perceptron is that $\mathbf{W}^{(0)}$ now has H *columns* – one weight vector per hidden neuron, computed in parallel via matrix multiplication.

Forward Propagation – Output Layer

The **output layer** contains one perceptron whose input is $\mathbf{A}^{(1)}$ (not $\mathbf{X}$):

$$\mathbf{z}_{T \times 1}^{(2)} = \mathbf{A}_{T \times H}^{(1)} \mathbf{w}_{H \times 1}^{(1)}, \quad z_t^{(2)} = a_{t1}^{(1)} w_1^{(1)} + a_{t2}^{(1)} w_2^{(1)}. \quad (15)$$

Applying Sigmoid gives the final prediction:

$$\hat{y}_t = a_t^{(2)} = \sigma(z_t^{(2)}), \quad t = 1, 2, 3, 4. \quad (16)$$

Key idea

Forward propagation is simply “*linear combination* $\rightarrow$ *activation*” applied **layer by layer**, with each layer’s output feeding the next layer’s input. Adding more layers/neurons only changes the matrix dimensions, not the pattern.

Backward Propagation – Setup

Exactly as before, the error, loss, and cost function are:

$$e_t = y_t - \hat{y}_t, \quad L_t = \frac{1}{2} e_t^2, \quad C = \sum_{t=1}^4 L_t = \frac{1}{2} \mathbf{e}^T \mathbf{e}. \quad (17)$$

Now, however, C depends on **two** sets of parameters – the output-layer weights $\mathbf{W}^{(1)}$ and the hidden-layer weights $\mathbf{W}^{(0)}$:

$$\min_{\mathbf{W}^{(0)}, \mathbf{W}^{(1)}} C \iff \frac{\partial C}{\partial \mathbf{W}^{(1)}} = \mathbf{0}_{H \times 1}, \quad \frac{\partial C}{\partial \mathbf{W}^{(0)}} = \mathbf{0}_{K \times H}. \quad (18)$$

Both are found via *gradient descent*, applying the chain rule **backward** from the output layer toward the input layer – hence the name *backpropagation*.

Gradient Descent for $\mathbf{W}^{(1)}$ (Output Layer)

Following the same chain-rule pattern as the single perceptron, with $\mathbf{A}^{(1)}$ as the input and $\mathbf{A}^{(2)}$ as the predicted output:

$$\mathbf{W}_{H \times 1}^{(1),(n+1)} = \mathbf{W}_{H \times 1}^{(1),(n)} + \alpha \mathbf{A}_{H \times T}^{(1)T} \delta_{T \times 1}^{(2),(n)}, \quad \delta^{(2),(n)} = -\mathbf{e}^{(n)} \circ \mathbf{A}^{(2),(n)} \circ (1 - \mathbf{A}^{(2),(n)}) \quad (19)$$

This is **identical in structure** to the perceptron update rule ($\mathbf{w}^{(n+1)} = \mathbf{w}^{(n)} + \alpha \mathbf{X}^T \{\dots\}$) – only the input matrix changed from $\mathbf{X}$ to $\mathbf{A}^{(1)}$, and it is transposed ($T \times H \rightarrow H \times T$) so the matrix dimensions match $\mathbf{W}^{(1)}$.

Gradient Descent for $\mathbf{W}^{(0)}$ (Hidden Layer)

Updating $\mathbf{W}^{(0)}$ requires the chain rule to reach further back, through the output layer's error *and* through the hidden layer's own activation:

$$\mathbf{W}_{K \times H}^{(0),(n+1)} = \mathbf{W}_{K \times H}^{(0),(n)} + \alpha \mathbf{X}_{K \times T}^T \boldsymbol{\delta}_{T \times H}^{(1),(n)} \quad (20)$$

$$\boldsymbol{\delta}_{T \times H}^{(1),(n)} = \boldsymbol{\delta}_{T \times 1}^{(2),(n)} \mathbf{W}_{1 \times H}^{(1),(n)T} \circ \mathbf{A}_{T \times H}^{(1),(n)} \circ (1 - \mathbf{A}^{(1),(n)})_{T \times H} \quad (21)$$

Interpretation

The error signal $\boldsymbol{\delta}^{(2)}$ from the output layer is *propagated backward* through $\mathbf{W}^{(1)}$ to become the error signal $\boldsymbol{\delta}^{(1)}$ at the hidden layer – this is literally where the term **“backpropagation”** comes from.

Summary of Gradient Descent Algorithm

Although the mathematical derivation for the artificial neural network model is more complicated than for the perceptron model, the Python code remains simple. To summarize the update rules for parameters $\mathbf{W}^{(1)}$ and $\mathbf{W}^{(0)}$ in backward propagation, the components for the *gradient descent* algorithm needed in the Python code are written again:

$$\mathbf{W}_{H \times 1}^{(1),(n+1)} = \mathbf{W}_{H \times 1}^{(1),(n)} + \alpha \cdot \mathbf{A}_{H \times T}^{(1),(n)T} \delta_{T \times 1}^{(2),(n)} \quad (22)$$

$$\mathbf{W}_{K \times H}^{(0),(n+1)} = \mathbf{W}_{K \times H}^{(0),(n)} + \alpha \cdot \mathbf{X}_{K \times T}^T \delta_{T \times H}^{(1),(n)} \quad (23)$$

With:

$$\delta_{T \times 1}^{(2),(n)} = -\mathbf{e}_{T \times 1}^{(n)} \circ \mathbf{A}_{T \times 1}^{(2),(n)} \circ (1 - \mathbf{A}_{T \times 1}^{(2),(n)}) \quad (24)$$

$$\delta_{T \times H}^{(1),(n)} = \delta_{T \times 1}^{(2),(n)} \mathbf{W}_{1 \times H}^{(1),(n)T} \circ \mathbf{A}_{T \times H}^{(1),(n)} \circ (1 - \mathbf{A}_{T \times H}^{(1),(n)}) \quad (25)$$

Although this *gradient descent* algorithm is derived for a *hidden layer* with two neurons, it applies just as well to a *hidden layer* with more than two neurons. In the Python code, adding or removing neurons only requires changing the value of H .

From Hand-Derived Gradients to Autograd

- For a 2-layer MLP, deriving $\delta^{(1)}$ and $\delta^{(2)}$ by hand is already tedious – for networks with dozens of layers (CNN, RNN, Transformer), it is impossible to do by hand.
- In practice, frameworks such as **PyTorch** and **TensorFlow/Keras** compute these gradients **automatically** using *automatic differentiation* (autograd), by applying the same chain rule we just derived, symbolically, on every operation in the network.
- As a practitioner, you define the **forward pass** (the architecture); the framework handles the **backward pass** (the gradients) for you.
- This is exactly what we will do in the live coding session: build the forward pass in a few lines of code and call `.backward()` / `model.fit()` to train.

Convolutional Neural Network (CNN)

MLP treats every pixel of an image as an independent feature – this ignores spatial structure and explodes in parameter count for large images. **CNN** solves this using two principles:

- **Locality**: a neuron only looks at a small, local patch of the image (not the whole image at once).
- **Translation invariance**: the same small filter (kernel) is reused (*shared weights*) across every location in the image.

The following slides build up the convolution operation from first principles to a full CNN.



From fully connected
to convolutions

Classifying Dogs and Cats in Images

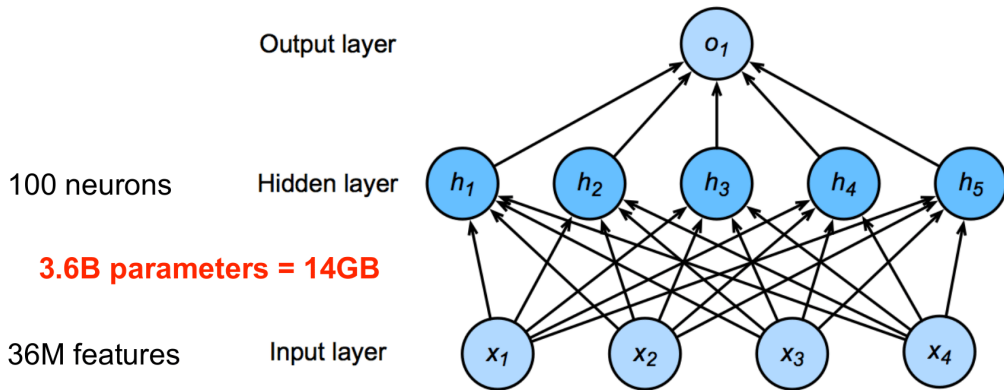
- Use a good camera
- RGB image has 36M elements
- The model size of a single hidden layer MLP with a 100 hidden size is 3.6 Billion parameters
- Exceeds the population of dogs and cats on earth (900M dogs + 600M cats)



Dual
12MP
wide-angle and
telephoto cameras



Flashback - Network with one hidden layer



$$\mathbf{h} = \sigma(\mathbf{W}\mathbf{x} + \mathbf{b})$$

Where is
Waldo?

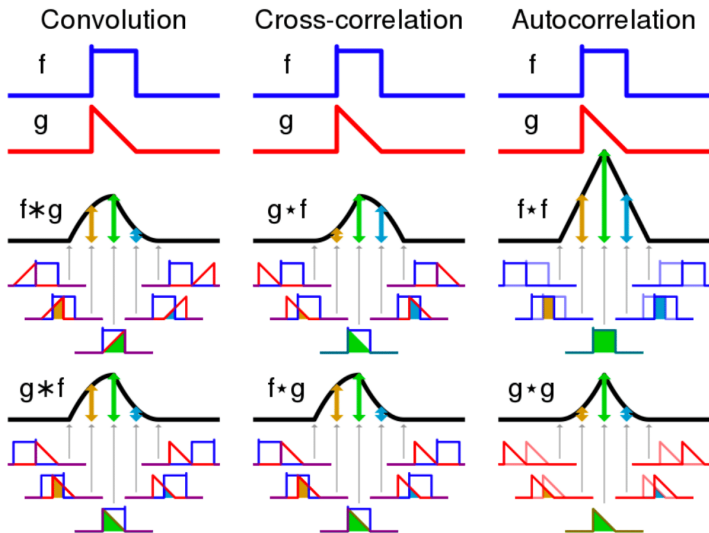


Two Principles

- Translation Invariance
- Locality



Convolution



2-D Cross Correlation

Input

0	1	2
3	4	5
6	7	8

Kernel

0	1
2	3

*

=

Output

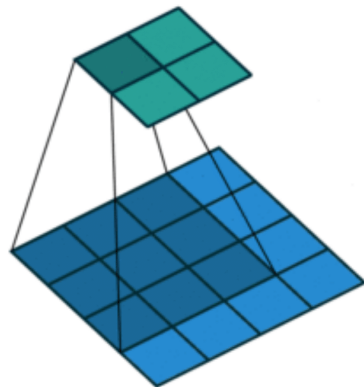
19	25
37	43

$$0 \times 0 + 1 \times 1 + 3 \times 2 + 4 \times 3 = 19,$$

$$1 \times 0 + 2 \times 1 + 4 \times 2 + 5 \times 3 = 25,$$

$$3 \times 0 + 4 \times 1 + 6 \times 2 + 7 \times 3 = 37,$$

$$4 \times 0 + 5 \times 1 + 7 \times 2 + 8 \times 3 = 43.$$



(vdumoulin@ Github)

2-D Convolution Layer

0	1	2
3	4	5
6	7	8

 *

0	1
2	3

 =

19	25
37	43

- $\mathbf{X} : n_h \times n_w$ input matrix
- $\mathbf{W} : k_h \times k_w$ kernel matrix
- b : scalar bias
- $\mathbf{Y} : (n_h - k_h + 1) \times (n_w - k_w + 1)$ output matrix

$$\mathbf{Y} = \mathbf{X} \star \mathbf{W} + b$$

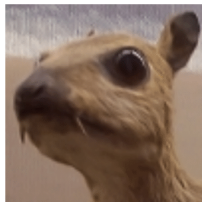
- $\mathbf{W}$ and b are learnable parameters

Examples

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

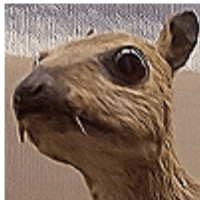


Edge Detection



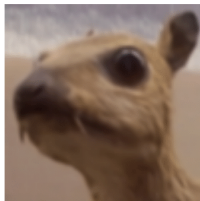
(wikipedia)

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

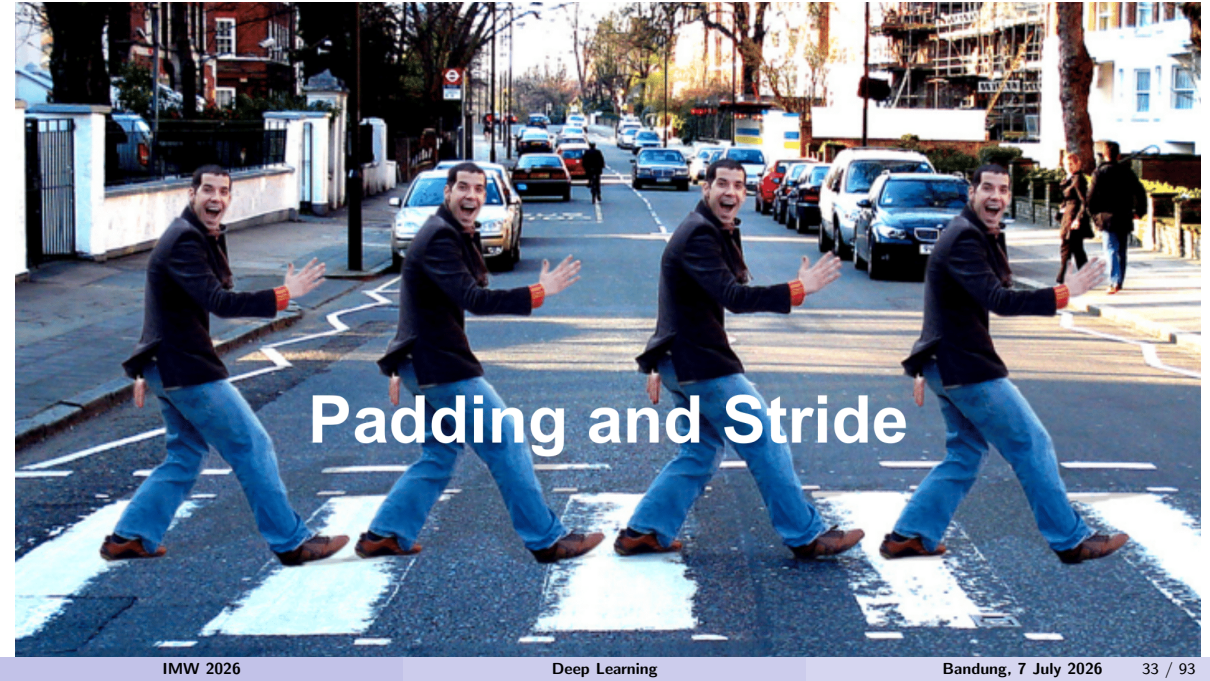


Sharpen

$$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$



Gaussian Blur

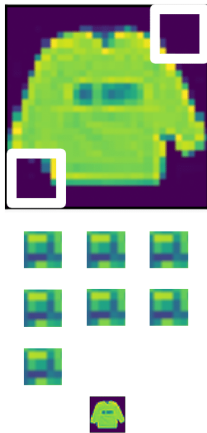


Padding and Stride

Padding

- Given a 32 x 32 input image
- Apply convolutional layer with 5 x 5 kernel
 - 28 x 28 output with 1 layer
 - 4 x 4 output with 7 layers
- Shape decreases faster with larger kernels
 - Shape reduces from $n_h \times n_w$ to

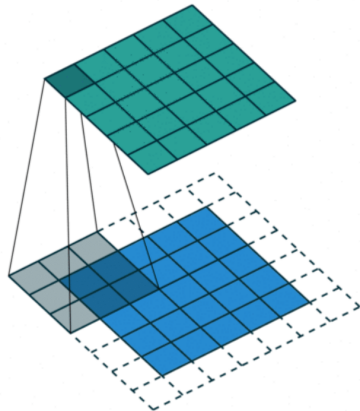
$$(n_h - k_h + 1) \times (n_w - k_w + 1)$$



Padding

Padding adds rows/columns around input

Input		Kernel		Output																																													
<table><tr><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>0</td><td>1</td><td>2</td><td>0</td></tr><tr><td>0</td><td>3</td><td>4</td><td>5</td><td>0</td></tr><tr><td>0</td><td>6</td><td>7</td><td>8</td><td>0</td></tr><tr><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td></tr></table>	0	0	0	0	0	0	0	1	2	0	0	3	4	5	0	0	6	7	8	0	0	0	0	0	0	*	<table><tr><td>0</td><td>1</td></tr><tr><td>2</td><td>3</td></tr></table>	0	1	2	3	=	<table><tr><td>0</td><td>3</td><td>8</td><td>4</td></tr><tr><td>9</td><td>19</td><td>25</td><td>10</td></tr><tr><td>21</td><td>37</td><td>43</td><td>16</td></tr><tr><td>6</td><td>7</td><td>8</td><td>0</td></tr></table>	0	3	8	4	9	19	25	10	21	37	43	16	6	7	8	0
0	0	0	0	0																																													
0	0	1	2	0																																													
0	3	4	5	0																																													
0	6	7	8	0																																													
0	0	0	0	0																																													
0	1																																																
2	3																																																
0	3	8	4																																														
9	19	25	10																																														
21	37	43	16																																														
6	7	8	0																																														



$$0 \times 0 + 0 \times 1 + 0 \times 2 + 0 \times 3 = 0$$

Padding

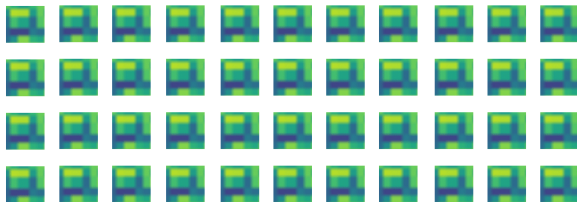
- Padding p_h rows and p_w columns, output shape will be

$$(n_h - k_h + p_h + 1) \times (n_w - k_w + p_w + 1)$$

- A common choice is $p_h = k_h - 1$ and $p_w = k_w - 1$
 - Odd k_h : pad $p_h/2$ on both sides
 - Even k_h : pad $\lceil p_h/2 \rceil$ on top, $\lfloor p_h/2 \rfloor$ on bottom

Stride

- Padding reduces shape linearly with #layers
 - Given a 224×224 input with a 5×5 kernel, needs 44 layers to reduce the shape to 4×4
 - Requires a large amount of computation



Stride

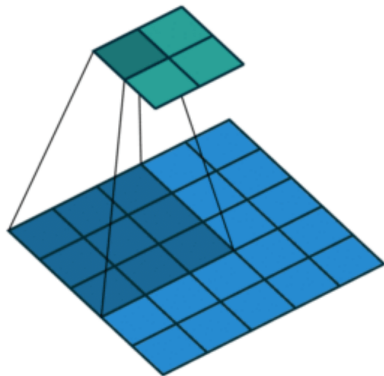
- Stride is the #rows/#columns per slide

Strides of 3 and 2 for height and width

Input		Kernel		Output																																	
<table><tr><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>0</td><td>1</td><td>2</td><td>0</td></tr><tr><td>0</td><td>3</td><td>4</td><td>5</td><td>0</td></tr><tr><td>0</td><td>6</td><td>7</td><td>8</td><td>0</td></tr><tr><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td></tr></table>	0	0	0	0	0	0	0	1	2	0	0	3	4	5	0	0	6	7	8	0	0	0	0	0	0	*	<table><tr><td>0</td><td>1</td></tr><tr><td>2</td><td>3</td></tr></table>	0	1	2	3	=	<table><tr><td>0</td><td>8</td></tr><tr><td>6</td><td>8</td></tr></table>	0	8	6	8
0	0	0	0	0																																	
0	0	1	2	0																																	
0	3	4	5	0																																	
0	6	7	8	0																																	
0	0	0	0	0																																	
0	1																																				
2	3																																				
0	8																																				
6	8																																				

$$0 \times 0 + 0 \times 1 + 1 \times 2 + 2 \times 3 = 8$$

$$0 \times 0 + 6 \times 1 + 0 \times 2 + 0 \times 3 = 6$$

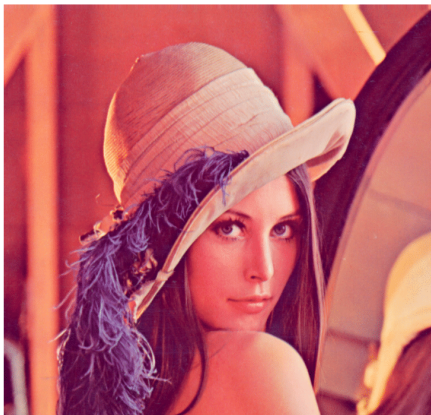




Multiple Input and Output Channels

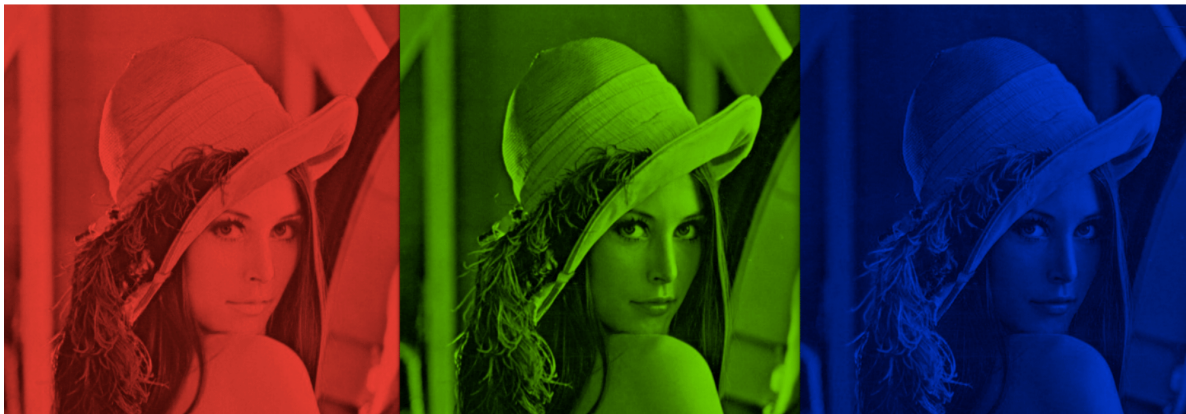
Multiple Input Channels

- Color image may have three RGB channels
- Converting to grayscale loses information



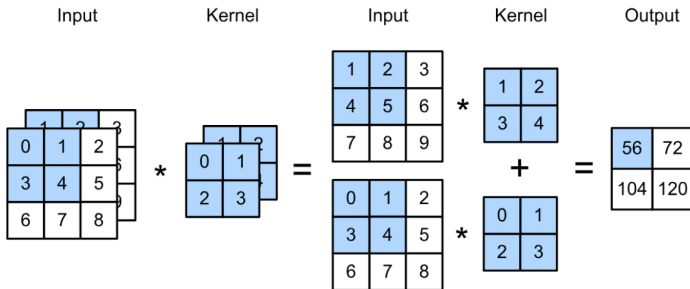
Multiple Input Channels

- Color image may have three RGB channels
- Converting to grayscale loses information



Multiple Input Channels

- Have a kernel for each channel, and then sum results over channels



$$\begin{aligned} &(1 \times 1 + 2 \times 2 + 4 \times 3 + 5 \times 4) \\ &+ (0 \times 0 + 1 \times 1 + 3 \times 2 + 4 \times 3) \\ &= 56 \end{aligned}$$

Multiple Input/Output Channels

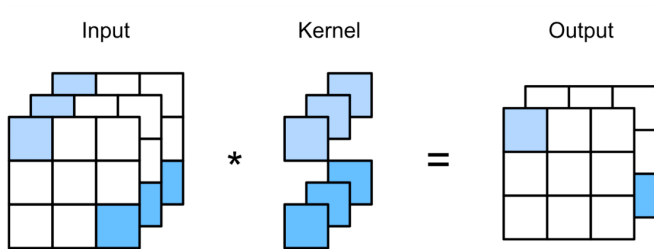
- Each output channel may recognize a particular pattern



- Input channels kernels recognize and combines patterns in inputs

1 x 1 Convolutional Layer

$k_h = k_w = 1$ is a popular choice. It doesn't recognize spatial patterns, but fuse channels.



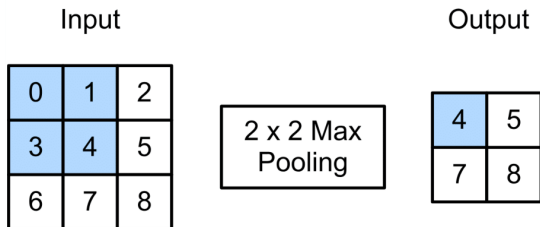
Equal to a dense layer with $n_h n_w \times c_i$ input and $c_o \times c_i$ weight.



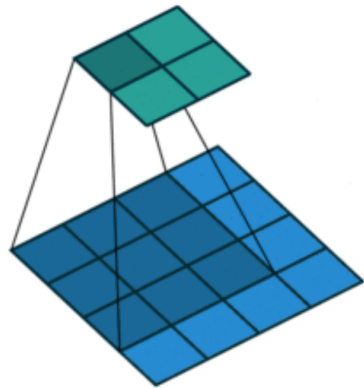
Pooling Layer

2-D Max Pooling

- Returns the maximal value in the sliding window

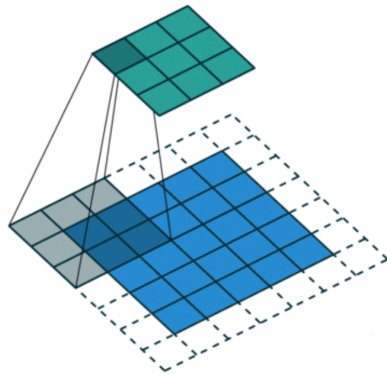


$$\max(0, 1, 3, 4) = 4$$



Padding, Stride, and Multiple Channels

- Pooling layers have similar padding and stride as convolutional layers
- No learnable parameters
- Apply pooling for each input channel to obtain the corresponding output channel

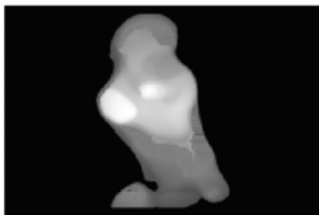


#output channels = #input channels

Average Pooling

- Max pooling: the strongest pattern signal in a window
- Average pooling: replace max with mean in max pooling
 - The average signal strength in a window

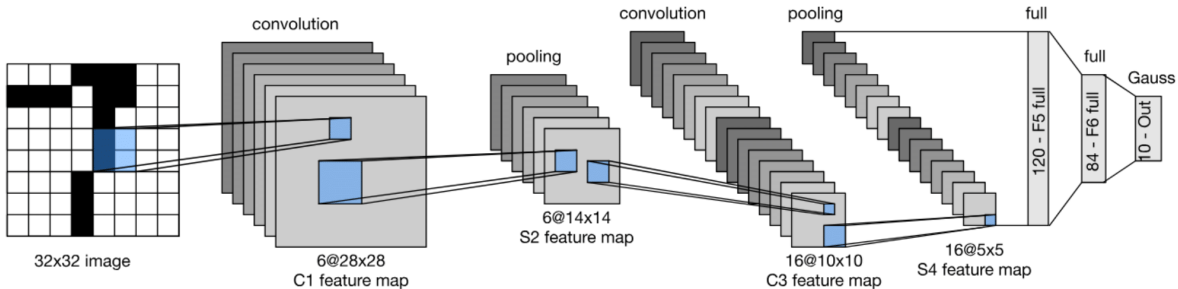
Max pooling



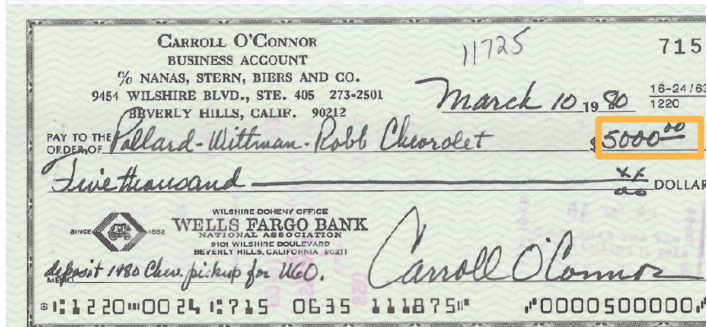
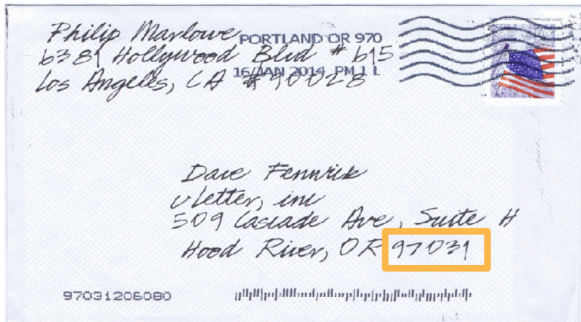
Average pooling



LeNet



Handwritten Digit Recognition



MNIST

- Centered and scaled
- 50,000 training data
- 10,000 test data
- 28 x 28 images
- 10 classes



Recurrent Neural Network (RNN) & LSTM

CNN and MLP assume inputs are independent of each other. But for **sequential data** (text, time series, audio), **order and context matter** – the meaning of a word depends on the words before it.

- **RNN**: processes a sequence one step at a time, carrying a *hidden state* that summarizes everything seen so far.
- **Problem**: over long sequences, gradients through many time steps tend to *vanish* or *explode*.
- **LSTM** (Long Short-Term Memory): adds gates (input, forget, output) that let the network learn what to remember and what to forget – solving the vanishing gradient problem.

Recurrent Neural Networks and Long Short-Term Memory

Sequence Modeling: Handwritten Text Translation

house → "house"

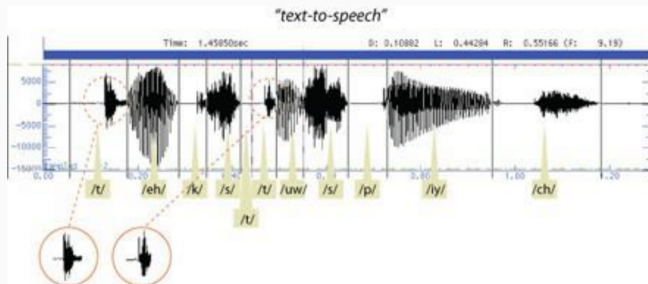
Winter is here. Go to
the store and buy some
snow shovels.

Winter is here. Go to the store and buy
some snow shovels.

- Input : Image
- Output: Text

<https://towardsdatascience.com/build-a-handwritten-text-recognition-system-using-tensorflow-2326a3487cd5>

Sequence Modeling: Text-to-Speech

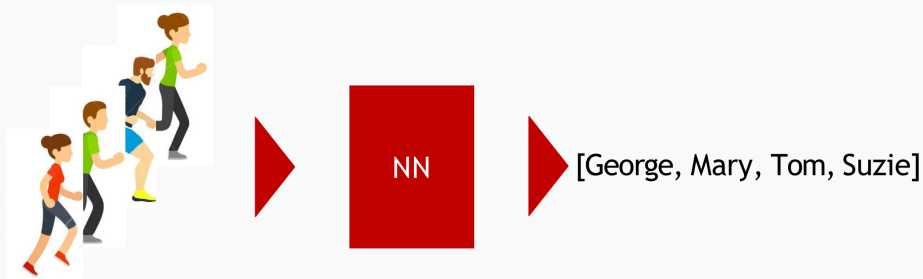


- Input : Audio
- Output: Text

Why RNNs

What can my NN do?

Training: Present to the NN examples and learn from them.



What can my NN do?

Prediction: Given an example



George

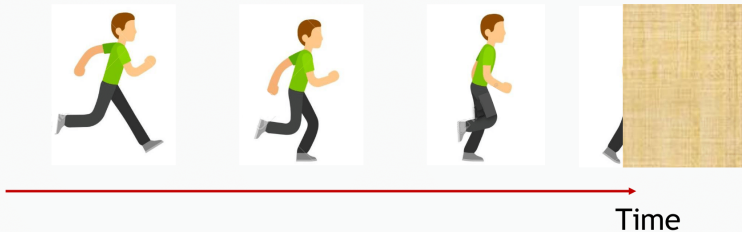


Mary

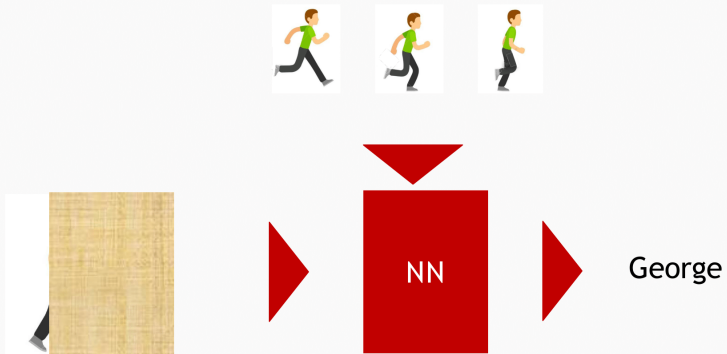
What my NN can NOT do?



Learn from previous examples

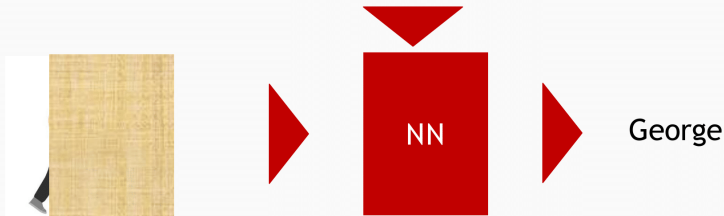


Recurrent Neural Network (RNN)



Recurrent Neural Network (RNN)

I have seen George
moving in this way
before.



RNNs recognize the data's sequential characteristics and use patterns to predict the next likely scenario.

Sequences

- We want a machine learning model to understand sequences, not isolated samples.
- Can MLP do this?
- Assume we have a sequence of temperature measurements and we want to take 3 sequential measurements and predict the next one

features

1

35

2

32

3

45

4

48

5

41

6

39

7

36

...

...

1

35

2

32

3

45

4

48

2

32

3

45

4

48

5

41

3

45

4

48

5

41

6

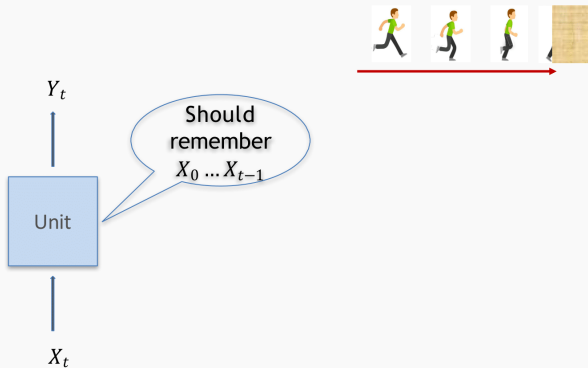
39

samples

Main Concept of RNNs

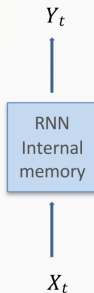
Memory

Somehow the computational unit should remember what it has seen before.



Memory

Somehow the computational unit should remember what it has seen before.
We'll call the information the unit's **state**.



Memory

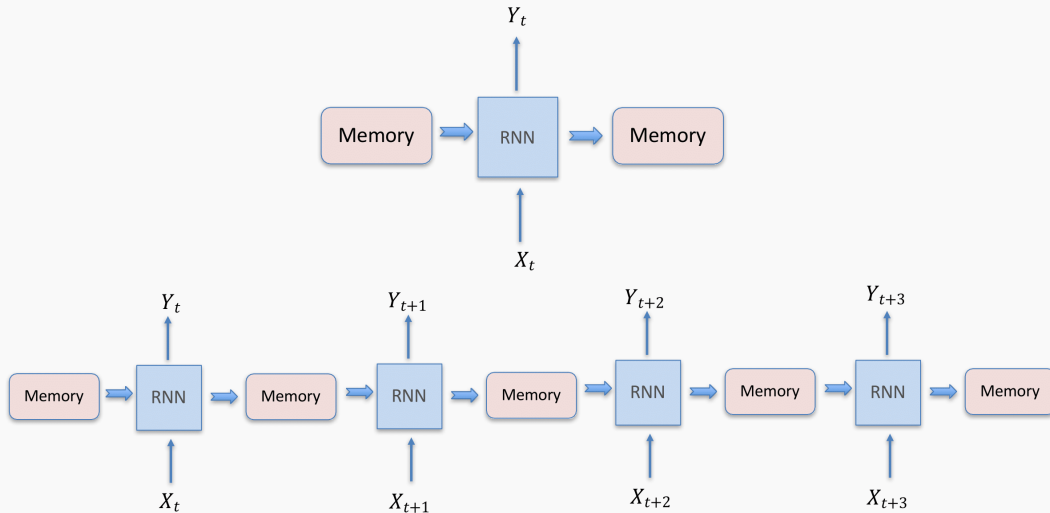
In neural networks, once training is over, the weights do not change. This means that the network is done learning and done changing.

Then, we feed in values, and it simply applies the operations that make up the network, using the values it has learned.

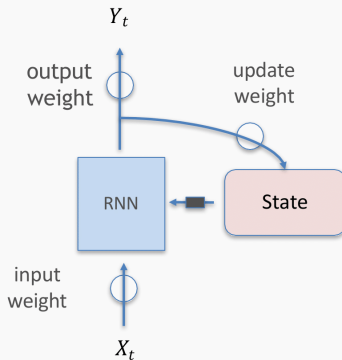
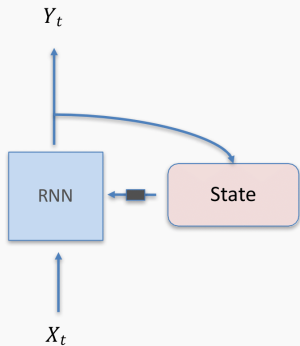
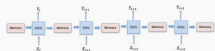
But the RNN units are able to remember new information after training has completed.

That is, they're able to keep changing after training is over.

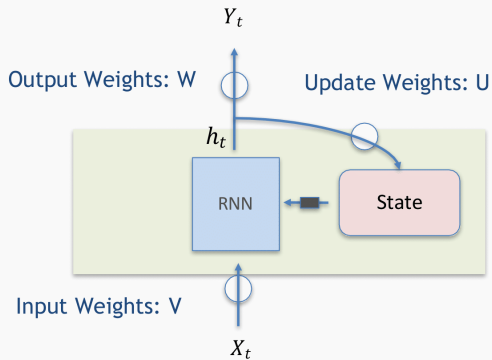
Building a RNN



Structure of an RNN cell



Backprop Through Time



Backprop Through Time

$$\hat{y}_t = g_y(Wh_t + b)$$

$$h_t = g_h(Vx_t + Uh_{t-1} + b')$$

$$\hat{y}_t = g_y(Wg_h(Vx_t + Uh_{t-1} + b') + b)$$

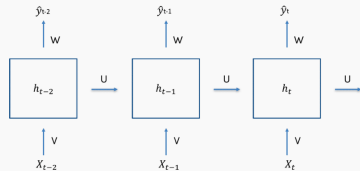
$$L = \sum_t L_t \quad L_t = L_t(\hat{y}_t)$$

$$\frac{dL}{dU} = \sum_t \frac{\partial L_t}{\partial \hat{y}_t} \frac{\partial \hat{y}_t}{\partial h_t} \frac{\partial h_t}{\partial U}$$

$$\frac{\partial h_t}{\partial U} = \sum_{k=1}^t \frac{\partial h_t}{\partial h_k} \frac{\partial h_k}{\partial U}$$

$$\frac{\partial h_t}{\partial h_k} = \frac{\partial h_t}{\partial h_{t-1}} \frac{\partial h_{t-1}}{\partial h_{t-2}} \dots \frac{\partial h_{k+1}}{\partial h_k} = \prod_{j=k+1}^t \frac{\partial h_j}{\partial h_{j-1}}$$

$$\frac{\partial L_t}{\partial U} = \frac{\partial L_t}{\partial \hat{y}_t} \frac{\partial \hat{y}_t}{\partial h_t} \left(\frac{dh_t}{dU} + \frac{dh_t}{dh_{t-1}} \frac{dh_{t-1}}{dU} + \frac{dh_t}{dh_{t-1}} \frac{dh_{t-1}}{dh_{t-2}} \frac{dh_{t-2}}{dU} + \dots \right)$$



$$\frac{\partial h_j}{\partial h_{j-1}} = g'_h U$$

Gradient Clipping

Prevents exploding gradients

Clip the norm of gradient before update.

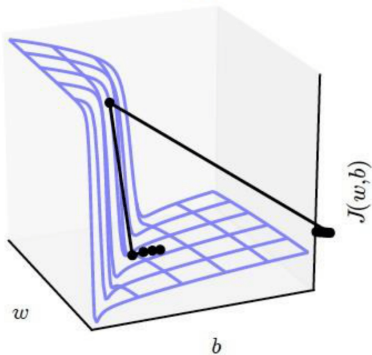
For some derivative g , and some threshold u

if $\|g\| > u$

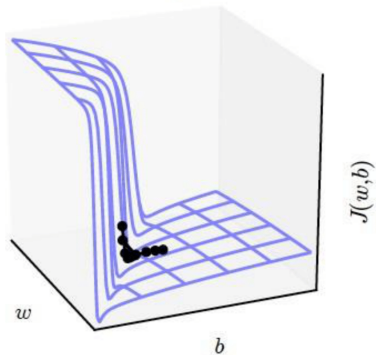
$$g \leftarrow \frac{gu}{\|g\|}$$

Gradient Clipping

Without clipping

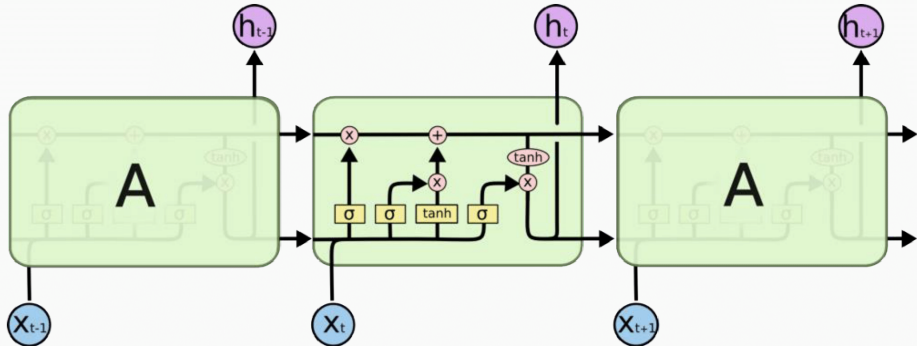


With clipping

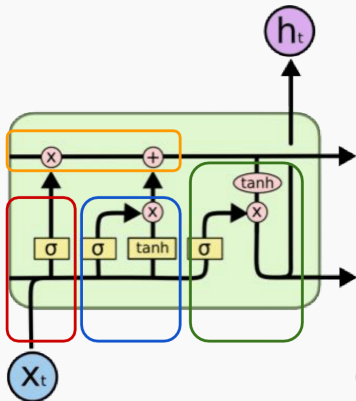


Gated RNN

LSTM: Long short term memory



Before to really understand LSTM lets see the big picture ...



Cell State

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

Output Gate

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh(C_t)$$

Forget Gate

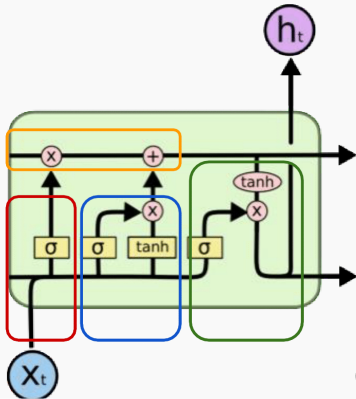
$$f_t = \sigma(W_f \cdot [h_{t-1}, x_{t-1}] + b_f)$$

Input Gate

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_f)$$

Before to really understand LSTM lets see the big picture ...



1. LSTM are recurrent neural network with a cell and a hidden state, boths of these are updated in each step and can be thought as memories.
2. Cell states work as a long term memory and the updates depends on the relation between the hidden state in $t-1$ and the input.
3. The hidden state of the next step is a transformation of the cell state and the output (which is the section that is in general used to calculate our loss, ie information that we want in a short memory).

From Data-Driven to Physics-Informed

So far, every network we trained learned purely from **labeled examples** $(\mathbf{x}_t, y_t)$ – MLP, CNN, and RNN/LSTM all minimize the distance between a prediction and an observed target.

But in many engineering and scientific problems:

- We already **know the governing equation** (an ODE or PDE) from physics.
- Labeled data is **sparse, expensive, or noisy** (e.g. a handful of sensor readings).
- We still want a model that is accurate *everywhere* in the domain, not just at the measured points.

Idea

What if the **loss function itself enforces the physics**, so the network is penalized whenever its output violates the known differential equation?

This is exactly what a **Physics-Informed Neural Network (PINN)** does.

Recap: Solving an ODE

Consider a first-order ODE with initial condition:

$$\frac{dy}{dt} = f(t, y), \quad y(0) = y_0. \quad (26)$$

Classical numerical solvers (Euler, Runge–Kutta):

- March forward in small time steps Δt , approximating $y(t + \Delta t)$ from $y(t)$.
- Very accurate for smooth, well-behaved problems, but tied to a *mesh/grid* of time steps.
- Do not naturally use extra measured data if it is available, and must be re-solved from scratch for every new initial condition or parameter value.

PINN's approach: treat $y(t)$ as a *function to be learned* by a neural network, trained so that it satisfies the ODE and the initial condition **everywhere in** t , not just on a grid.

The Core Idea of a PINN

Let $\hat{y}(t; \theta)$ be a small MLP with input t , output $\hat{y}$, and parameters θ (weights and biases).

- 1 **Ansatz:** treat $\hat{y}(t; \theta)$ as our candidate solution to the ODE.
- 2 **Physics residual:** plug $\hat{y}(t; \theta)$ into the ODE. If $\hat{y}$ were the exact solution, the residual would be exactly zero:

$$r(t; \theta) = \frac{d\hat{y}}{dt}(t; \theta) - f(t, \hat{y}(t; \theta)). \quad (27)$$

- 3 **Loss:** train θ to make $r(t; \theta)$ close to zero at many points t , **and** to satisfy $\hat{y}(0; \theta) = y_0$.

Key trick

$\frac{d\hat{y}}{dt}$ is computed **exactly** using **automatic differentiation** – the same machinery from the “Autograd” slide, but now differentiating the network's output with respect to its **input** t , not with respect to θ .

Worked Example: Exponential Decay

As a concrete running example, consider radioactive decay / cooling:

$$\frac{dy}{dt} = -k y(t), \quad y(0) = y_0, \quad t \in [0, T]. \quad (28)$$

This ODE has a known closed-form solution, which lets us **check** the PINN against ground truth:

$$y(t) = y_0 e^{-kt}. \quad (29)$$

We will train a network $\hat{y}(t; \theta)$ to approximate this solution **using only**:

- the ODE itself (k is known), and
- the single initial condition $y(0) = y_0$ –

with **no** labeled (t_i, y_i) pairs from the true solution at all.

Step 1 – The Network as a Solution Ansatz

A small feed-forward network maps the scalar time t to a scalar prediction $\hat{y}$:

$$t \longrightarrow \boxed{\text{Dense} \rightarrow \tanh \rightarrow \text{Dense} \rightarrow \tanh \rightarrow \text{Dense}} \longrightarrow \hat{y}(t; \theta). \quad (30)$$

- Same structure as the MLPs from earlier in this session – linear combination, then activation, layer by layer.
- $\tanh$ is preferred over ReLU here: the residual needs $\hat{y}$ to be **smooth and differentiable**, and $\tanh$ gives a smooth derivative everywhere.
- Unlike a classification/regression MLP, there is no dataset of (t, y) pairs to fit directly – t is just *sampled*, not measured.

Step 2 – The Physics Residual via Autodiff

For the decay equation, the residual function is:

$$r(t; \theta) = \frac{d\hat{y}}{dt}(t; \theta) + k \hat{y}(t; \theta). \quad (31)$$

To compute $\frac{d\hat{y}}{dt}$, we do **not** derive a formula by hand. Instead:

- 1 Feed t through the network to get $\hat{y}(t; \theta)$ (forward pass).
- 2 Ask the automatic-differentiation engine (`tf.GradientTape` / `torch.autograd`) for $\frac{\partial \hat{y}}{\partial t}$ – exactly the same chain-rule machinery used for backpropagation, just differentiating with respect to the **input** instead of the **weights**.
- 3 Combine the result into $r(t; \theta)$ above.

If $\hat{y}$ solves the ODE exactly at t , then $r(t; \theta) = 0$.

Step 3 – The PINN Loss Function

Sample a set of **collocation points** $\{t_i\}_{i=1}^N$ across the domain $[0, T]$ (e.g. randomly, or on a grid – no data required, just points in time).

$$\mathcal{L}_{\text{physics}}(\theta) = \frac{1}{N} \sum_{i=1}^N r(t_i; \theta)^2 \quad (32)$$

$$\mathcal{L}_{\text{IC}}(\theta) = (\hat{y}(0; \theta) - y_0)^2 \quad (33)$$

$$\mathcal{L}(\theta) = \mathcal{L}_{\text{physics}}(\theta) + \lambda \mathcal{L}_{\text{IC}}(\theta) \quad (34)$$

- $\mathcal{L}_{\text{physics}}$ pushes $\hat{y}$ to **obey the ODE** at every collocation point.
- $\mathcal{L}_{\text{IC}}$ anchors the solution to the correct **starting value** – without it, $\hat{y} \equiv 0$ would trivially satisfy the ODE!
- λ is a weight balancing the two terms (a common practical tuning knob in PINNs).

Step 4 – Training Algorithm

The training loop looks exactly like the gradient descent we derived earlier, with one extra layer of automatic differentiation:

- 1 Sample a batch of collocation points $\{t_i\}$ in $[0, T]$ (mesh-free – resampled every iteration).
- 2 **Forward pass:** compute $\hat{y}(t_i; \theta)$ for all i , plus $\hat{y}(0; \theta)$.
- 3 **Inner autodiff:** compute $\frac{d\hat{y}}{dt}(t_i; \theta)$ to build the residual $r(t_i; \theta)$.
- 4 Compute the total loss $\mathcal{L}(\theta) = \mathcal{L}_{\text{physics}} + \lambda \mathcal{L}_{\text{IC}}$.
- 5 **Outer autodiff:** compute $\frac{\partial \mathcal{L}}{\partial \theta}$ (ordinary backpropagation).
- 6 Update $\theta \leftarrow \theta - \alpha \frac{\partial \mathcal{L}}{\partial \theta}$ (Adam, as usual).

Note

Two nested uses of automatic differentiation: once with respect to t (physics), once with respect to θ (learning).

Why This Works Without Labeled Data

- A classical supervised network needs many (t_i, y_i) pairs of the **true solution** to fit – exactly what we usually **don't have** for a real physical system.
- A PINN instead needs only:
 - the **equation itself** (already known from physics/engineering), and
 - the **initial/boundary conditions** (typically 1–2 known values).
- Collocation points t_i are *free* to generate – they are just numbers in the domain, not measurements.
- If a few real measurements *are* available, they can be added as an **extra data-loss term**, blending known physics with observed data.

This is the paradigm shift

Ordinary deep learning: learn a function from data. PINN: learn a function from an **equation**, with data as an optional bonus.

Extending to Second-Order ODEs

The same idea extends directly to a damped harmonic oscillator:

$$m \frac{d^2 y}{dt^2} + c \frac{dy}{dt} + k y = 0, \quad y(0) = y_0, \quad \dot{y}(0) = v_0. \quad (35)$$

- The residual now needs $\frac{d^2 \hat{y}}{dt^2}$ – obtained by applying automatic differentiation **twice** in a row (differentiate the derivative again).
- Two initial conditions ($y(0) = y_0$ and $\dot{y}(0) = v_0$) both become extra loss terms, exactly like $\mathcal{L}_{IC}$ before.
- Nothing else in the training loop changes – same collocation-point sampling, same Adam optimizer, same nested-autodiff pattern.

This is how PINNs scale to higher-order ODEs, systems of ODEs, and eventually **PDEs** (adding spatial derivatives the same way).

Strengths and Limitations of PINNs

Strengths

- Mesh-free – no grid to design or refine.
- Works with very little (or zero) labeled data.
- Naturally handles **inverse problems**: make an unknown physical constant (e.g. k) trainable, and let data loss recover it.
- One trained network can query the solution at *any* t , not just grid points.

Limitations

- Training can be slow and sensitive to how $\mathcal{L}_{\text{physics}}$ and $\mathcal{L}_{\text{IC}}$ are weighted.
- Struggles with stiff, chaotic, or very high-frequency solutions (“spectral bias” of neural networks).
- No convergence guarantees like classical numerical solvers have.
- Usually slower than a classical solver for problems that are already easy to mesh.

- Introduced by Raissi, Perdikaris & Karniadakis (2019) as a general framework for both **forward** problems (solve the equation) and **inverse** problems (discover unknown parameters/equations from data).
- Widely used today in computational physics, fluid dynamics, structural engineering, and **digital twins** – wherever a physical law is known but full-field data is not.
- Same recipe generalizes from ODEs (1 input: t) to PDEs (inputs: $t, x, y, z, \dots$) by adding the relevant partial derivatives via autodiff.

Next

In the live coding session, we will implement exactly the exponential-decay example above end-to-end in TensorFlow, and compare the trained PINN against the known closed-form solution $y_0 e^{-kt}$.

When to Use What?

- **MLP** – tabular/structured data: churn prediction, credit scoring, simple regression/classification on rows of features.
- **CNN** – grid-like data: image classification, object detection, medical imaging; also used on 1D grids (e.g., signal/sensor data).
- **RNN/LSTM** – sequential data: time series forecasting, sensor logs, older text/speech models.
- **PINN** – when you know the governing ODE/PDE but have sparse or noisy data: engineering simulations, digital twins, inverse parameter estimation.
- **Transformer** (modern extension, not covered in depth today) – has largely replaced RNN/LSTM for text (ChatGPT-style LLMs) and is now competitive for vision and time series too, via the *attention* mechanism.

Rule of thumb

Start simple: if a well-tuned MLP or classical ML model (e.g. gradient boosting) already solves your problem, you may not need a deeper architecture at all.

Comparison of MLP, CNN, RNN, and LSTM

Aspect	MLP (Multilayer Perceptron)	CNN (Convolutional Neural Network)	RNN (Recurrent Neural Network)	LSTM (Long Short-Term Memory)
Input Type	Fixed-size vectors	Grid-like data (e.g., images)	Sequential data	Sequences with long-term dependencies
Architecture	Fully connected layers	Convolution + pooling layers	Recurrent time-step connections	Recurrent with memory gates (input, forget, output)
Strengths	Simple and effective on structured/tabular data	Captures spatial patterns and translation invariance	Remembers recent inputs in sequence	Preserves long-term memory, solves vanishing gradient
Limitations	Ignores spatial/temporal structure	Not suited for sequence data	Struggles with long dependencies	Higher complexity and training time
Training Method	Backpropagation + gradient descent	Backpropagation with weight sharing	Backpropagation Through Time (BPTT)	BPTT + gated memory updates
Use Cases	Classification, regression	Image classification, object detection	Text prediction, signal modeling	Language modeling, translation, speech recognition

Next: Live Coding Session

- Tools: Python, PyTorch / Keras/TensorFlow, Google Colab (free GPU).
- Demo plan (2 hours):
 - 1 Build an MLP from scratch for a simple tabular dataset.
 - 2 Train a small CNN for image classification (e.g., MNIST/Fashion-MNIST).
 - 3 Train a simple RNN/LSTM on sequential data (e.g., time-series forecasting).
 - 4 Build a PINN to solve the exponential-decay ODE and compare against the analytical solution.
 - 5 Compare results and discuss best practices (overfitting, learning rate, etc.).

References

- ① Dive into Deep Learning Dive into Deep Learning 1.0.3 documentation <https://d2l.ai/>
- ② Chollet, Francois, Deep Learning with Python, Manning Publications Co., 2018
- ③ Ian Goodfellow, Yoshua Bengio, and Aaron Courville, Deep Learning, The MIT Press, 2016
- ④ Trask, Andrew. *A Neural Network in 13 Lines of Python (Part 2 - Gradient Descent)*.
<https://iamtrask.github.io/2015/07/27/python-network-part2/>
- ⑤ Spencer-Harper, Milo. *How to Build a Simple Neural Network in 9 Lines of Python Code*.
[https://medium.com/technology-invention-and-more/
how-to-build-a-simple-neural-network-in-9-lines-of-python-code-cc8f23647ca1](https://medium.com/technology-invention-and-more/how-to-build-a-simple-neural-network-in-9-lines-of-python-code-cc8f23647ca1)
- ⑥ Karpathy, Andrej. *Yes, You Should Understand Backprop*.
<https://medium.com/@karpathy/yes-you-should-understand-backprop-e2f06e4b496b>
- ⑦ Muhammad Syamsuddin. *Matematika untuk Deep Learning (Bagian 1 dan 2)*. FMIPA ITB, 2025.
- ⑧ Raissi, M., Perdikaris, P., & Karniadakis, G.E. *Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations*. Journal of Computational Physics, 378, 686–707, 2019.